

Recall & Warranty Guardian

The architecture: gren graphs on the Strands Agents SDK, Claude through the Strands model provider (the Anthropic API today, Amazon Bedrock when the account is authorized), one human gate, one household.

AGENTS FOR HUMANS · EVERYDAY AGENTS · STRANDS AGENTS SDK · APACHE-2.0

github.com/KrishGup/Recall-and-Warranty-Guardian · live: guardian.44-214-230-44.sslip.io

Recalls reach everyone and match nobody

PROBLEM

- Regulators publish thousands of recalls each year. CPSC, NHTSA, and openFDA each have their own feed.
- The notices go to everyone. Nobody matches them to the products in one home.
- The remedies are free. Most of them stay unclaimed. Warranties expire with claims never filed.
- A person does not want another app to check. A person wants one message, one decision, only when it matters.

SOLUTION

- Guardian builds an inventory of the household from forwarded receipts, order emails, and VINs.
- Each night, a Strands graph pulls the new recalls, matches them in code, and gives only the ambiguous pairs to a model.
- A triage agent and an adversarial verifier decide what deserves an interrupt. The graph stops at one human gate.
- After one approval, a side-effect node sends the remedy request with the receipt attached. Then it schedules a follow-up.

594

CPSC recalls in one 400-day window

1,200

openFDA enforcement reports in the same window

0

model calls on a quiet night

What changes for the household

BEFORE

- The household hears about a recall by chance, or never.
- A recall notice needs a manual search of the model number, the UPC, and the purchase date.
- A remedy request needs the receipt, the contact address, and a letter. Most people stop here.
- Nobody tracks the warranty window of a blender bought in 2016.

AFTER

- Every night, the inventory is compared with the new recalls. The household sees a quiet score, not a feed.
- A critical hazard becomes one SMS with one decision. A standard hazard goes to a weekly digest.
- One tap files the remedy request with the receipt. A follow-up runs after 10 business days.
- A warranty that ends soon triggers one question: is anything wrong with it?

1

message for each affected item

72 h

a gate holds a decision, then it expires

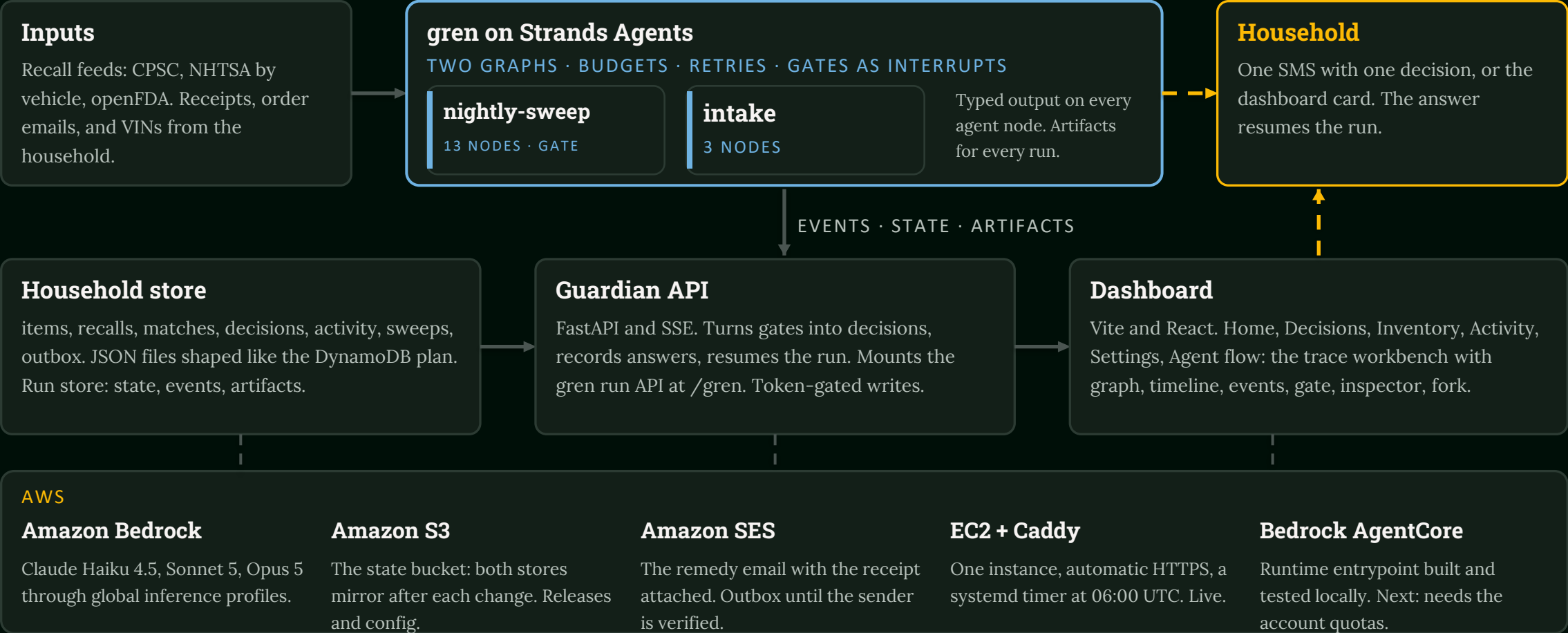
10 d

business days to the follow-up

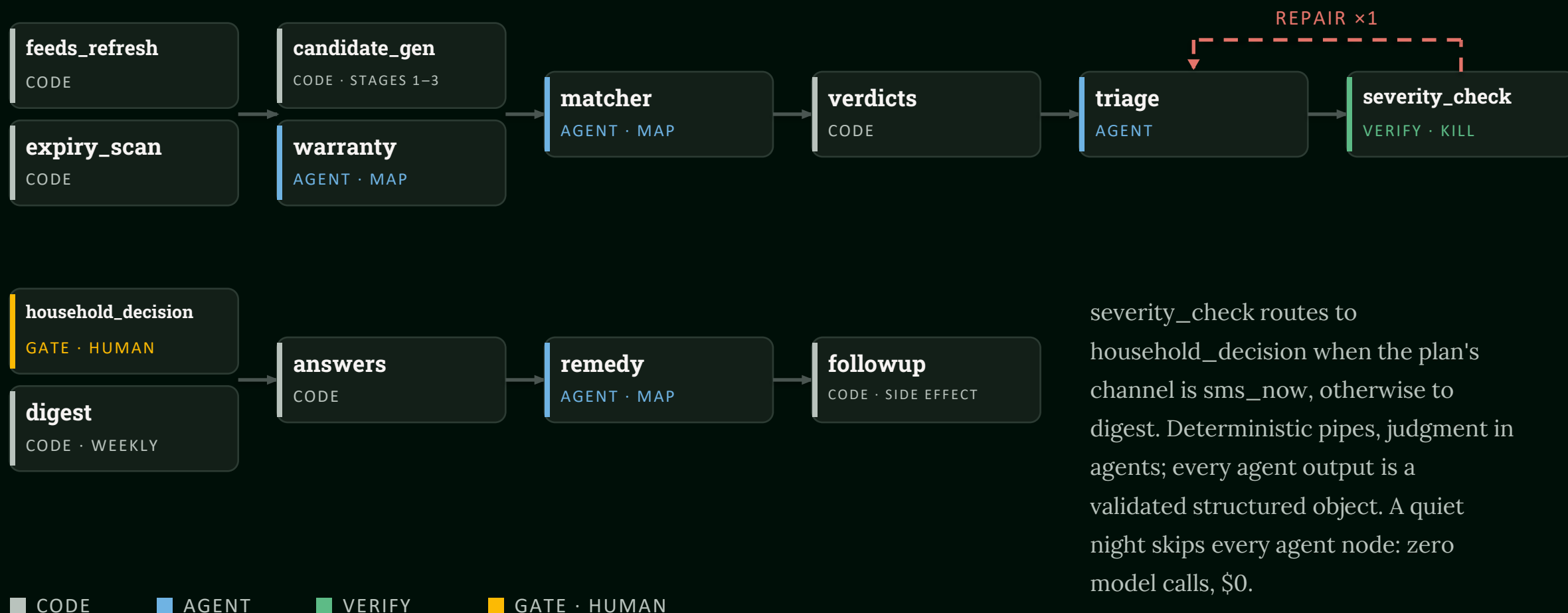
Four modes, one graph runtime

MODE	TRIGGER	WHAT IT DOES	OUTPUT
Nightly sweep	systemd timer at 06:00 UTC, the dashboard logo, or guardian sweep	Pulls CPSC, NHTSA, and openFDA. Matches in code (UPC, VIN, model, lexical, sold window). A model adjudicates the ambiguous pairs. Triage plans the night. A verifier checks the plan.	Decisions for the household, a digest, a sweep record, the activity log
Intake	A forwarded receipt, a pasted order email, or a VIN	Redacts card numbers. Extracts brand, model, UPC, purchase date, retailer, price, and warranty term. Decodes a VIN with NHTSA vPIC.	One inventory item with a warranty window
Household decision	The gate pauses the run. An SMS link or the dashboard opens the card.	Shows one card for each affected item: request remedy, no longer own, not mine, snooze. Batches the answers. Releases the gate on the first approval.	A gate approval with the answers as JSON
Remedy and follow-up	An approved decision	Writes the request from the remedy text and the contact address of the recall. Sends it one time. Records the action. Schedules the follow-up.	An email (SES or outbox), an action record, a follow-up date
Warranty check-in	A warranty window that ends within the threshold	Asks one question at the right time. A reported problem becomes a claim decision.	A decision card, or silence
Agent flow	Any run, live or past	Shows the run as the graph executed it: canvas, timeline, events, metrics, gate, spec, and each node. Forks a run from any node.	The trace workbench in the dashboard

One household, two graphs, one human gate



The nightly sweep: 13 nodes on Strands Agents



severity_check routes to **household_decision** when the plan's channel is **sms_now**, otherwise to **digest**. Deterministic pipes, judgment in agents; every agent output is a validated structured object. A quiet night skips every agent node: zero model calls, \$0.

How a gren graph runs on Strands Agents

GRAPHBUILDER

Every node is a Strands MultiAgentBase executor. gren derives the edges from data references such as `$nodes.matcher.outputs`. Nobody declares an edge by hand.

AND JOINS

Strands readiness is OR. gren adds an edge condition, so a node runs only when all of its dependencies are complete.

STRUCTURED OUTPUT

An agent node runs a Strands Agent with `structured_output_model`. gren validates the output against the JSON schema and makes one bounded repair call on an invalid output.

GATES ARE INTERRUPTS

A `BeforeNodeCallEvent` hook raises `event.interrupt`. The run pauses. The checkpoint is on disk. The process can exit. Resume replays completed nodes with no new model calls.

VERIFIER WITH KILL AUTHORITY

`severity_check` returns {`verdict`, `reasons`, `confidence`}. A rejection resets triage with the reasons as feedback through a conditional back-edge, at most one round.

BUDGETS AND SIDE EFFECTS

The engine enforces retries, the fallback model, timeouts, the quorum on fan-outs, and the spend cap. A side-effect node runs one time and only behind an approval.

Every node writes an artifact (system prompt, prompt, schema, raw output, usage, cost) under `var/runs/<run>/artifacts`. `events.jsonl` is the trace that the dashboard streams. See ARCHITECTURE.md section 1.

What runs where: the agent

COMPONENT	TECHNOLOGY	DESCRIPTION
gren runtime	Strands Agents SDK, Python	Compiles a YAML graph into a Strands Graph. Executors, AND joins, budgets, retries, gates, artifacts, events.
nightly-sweep graph	gren spec, 13 nodes	feeds_refresh, expiry_scan, candidate_gen, warranty, matcher, verdicts, triage, severity_check, household_decision, digest, answers, remedy, followup.
intake graph	gren spec, 3 nodes	redact (code), extract (agent, Haiku), save_item (code). A VIN goes through the NHTSA vPIC decoder.
Feeds	httpx clients	CPSC SaferProducts API, NHTSA recalls by vehicle, openFDA food enforcement. Recorded fixtures replay in the tests.
Matching	rapidfuzz, code	Stage 1 exact keys (UPC, vehicle, model). Stage 2 lexical score 80 or more with brand overlap. Stage 3 sold window of plus or minus 60 days. Stage 4 the matcher agent.
Policy	code	Severity keyword pass, notification budget, warranty windows, quiet categories.
Models	Claude (Anthropic API today; Amazon Bedrock pending)	Haiku 4.5 for extraction and verification. Opus 5 for the matcher and triage. Sonnet 5 for the remedy text. Global inference profiles.

What runs where: the household layer and the surfaces

COMPONENT	TECHNOLOGY	DESCRIPTION
Guardian service	Python	Turns gate.waiting into Decision rows and SMS. Records answers. Writes the gate approval. Resumes the run. Formats the activity log.
Household store	JSON files, DynamoDB-shaped	items, recalls, matches, decisions, activity, sweeps, outbox, prefs, household. One file per entity under var/household.
Notifier	SNS, SES, outbox	SMS through SNS and email through SES when configured. Otherwise the outbox folder.
Guardian API	FastAPI, SSE	/api for the dashboard, /gren/api for the run workbench, /api/events for live updates. Token gate on writes.
Dashboard	Vite, React, TypeScript	Home, Decisions, Inventory, Activity, Settings, Agent flow. Dark, RTL, mobile. Three fonts, five brand colors.
Live deployment	EC2, Caddy, systemd, S3	One instance with automatic HTTPS. A timer runs the sweep at 06:00 UTC. The store mirrors to the state bucket.
AgentCore path	Bedrock AgentCore Runtime	BedrockAgentCoreApp entrypoint with invoke kinds sweep, answer, intake, status. Built and tested locally. Blocked by account quotas of 0.

One night, step by step: feeds to the verifier

STEP	DESCRIPTION
1	The systemd timer posts /api/sweep at 06:00 UTC. The service checks the daily budget and starts the nightly-sweep graph with the window and the trigger.
2	feeds_refresh pulls the new records from CPSC, NHTSA (one query for each vehicle), and openFDA. It normalizes them into RecallRecord rows. expiry_scan reads the warranty windows and the notification budget.
3	candidate_gen compares every item with every recall in code: exact keys, lexical similarity with brand overlap, and the sold window. It emits certain matches and candidate pairs.
4	matcher (agent, map) adjudicates each candidate pair with a MatchVerdict. warranty (agent, map) writes the check-in questions for the windows that end soon.
5	verdicts merges the certain matches, the verdicts, and the questions into one list.
6	trriage (agent) applies the severity rules and the budget. It returns a SurfacePlan: the channel, the severity, the decisions, and the SMS text.
7	severity_check (verify) compares the plan with the deterministic severity pass. A rejection sends triage back one time with the reasons.

One night, step by step: the gate to the follow-up

STEP	DESCRIPTION
8	The plan routes to household_decision when the channel is sms_now. Otherwise digest records the standard hazards for the weekly digest and the run completes.
9	The gate raises a Strands interrupt. The run pauses on disk. The service writes one Decision row for each item, sends the SMS, and marks the match as surfaced.
10	The household answers by the SMS link, the dashboard, or the CLI. The service records the answer. On the first approval, it writes the gate approval with all answers as JSON and resumes the run.
11	answers turns the approval into the list of approved decisions. remedy (agent, map) writes an ActionReport for each one.
12	followup sends each request one time (SES or outbox), records the action, closes the match, and schedules the follow-up after 10 business days.
13	Every event goes to events.jsonl and to /api/events. The dashboard updates over SSE. The store mirrors to S3 after each change.

Three stores, one bucket

HOUSEHOLD STORE

- items: brand, model, UPC, purchase date, retailer, price, warranty term, VIN.
- recalls: normalized records from the three feeds, with the severity pass.
- matches: item, recall, stage, score, verdict, and the state: candidate, confirmed, surfaced, closed.
- decisions: headline, facts, options, state, answer, outcome, run id.
- activity, sweeps, outbox, prefs, household.

RUN STORE (GREN)

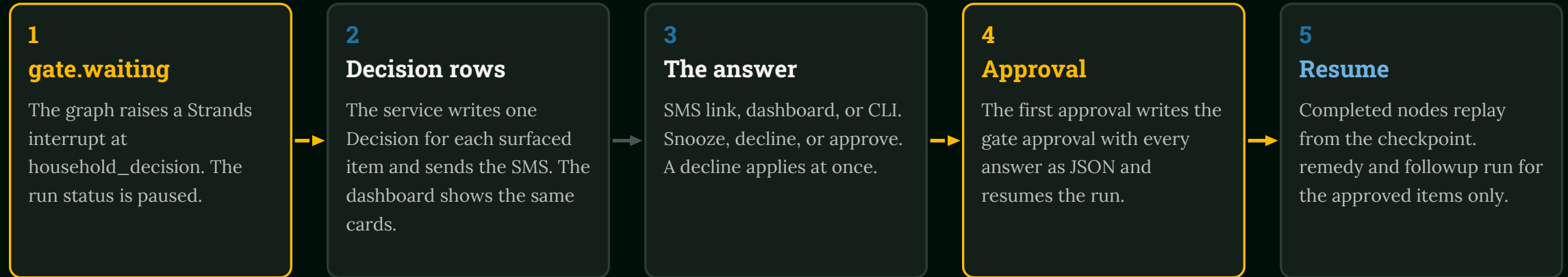
- state.json: the run, its input, the status of each node, totals, approvals.
- events.jsonl: the trace. The dashboard streams it.
- artifacts/: the prompt, the schema, the raw output, usage, and cost of every model call.
- tasks/ and approvals/: the gate records that resume a paused run.

SYNC AND CONFIG

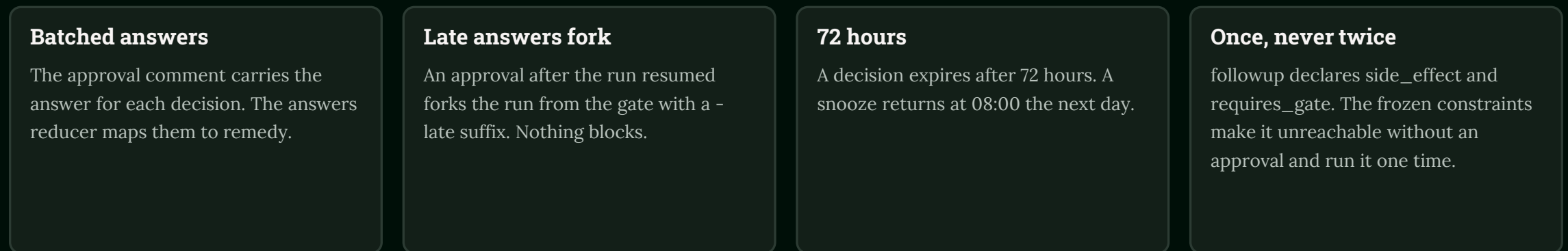
- GUARDIAN_STATE_SYNC=1 mirrors both stores to the S3 state bucket after each change. A new instance pulls them at start.
- config/env in the bucket holds the runtime environment of the server.
- The JSON store follows the single-table DynamoDB design of the plan. A move to DynamoDB is a driver change.

THE HUMAN GATE

One interrupt, durable on disk



RULES THE ENGINE ENFORCES



Cents per night, capped three times

CONTROL	SCOPE	VALUE AND EFFECT
Quiet night	the graph	Zero model calls. The when conditions skip matcher, triage, and every node after them.
Model per node	the spec	Haiku 4.5 for extraction and verification. Opus 5 for the matcher and triage. Sonnet 5 for the remedy. A typical night with one ambiguous pair makes 4 to 6 calls.
gren budget.max_cost_usd	one run	\$3.00. The engine fails the run when its own recorded spend exceeds the cap.
GUARDIAN_DAILY_BUDGET_USD	the household	\$10 by default. The service refuses to start a sweep or an intake once the recorded spend of the day reaches it.
AWS Budgets	the account	A \$50 monthly alert. AWS Budgets alert; they do not cap. A budget action can restrict an IAM user, never the root user.
First live sweep	measured	594 CPSC recalls, 3 NHTSA campaigns, 1,200 openFDA reports. 7,143 pairs in code. 3 pairs to the model. 235 s. \$0.41.
Live instance	EC2 t3.small	About \$0.60 a day. deploy_ec2.py down removes everything but the bucket.
Token gate	the public dashboard	GUARDIAN_API_TOKEN gates every POST, PUT, and DELETE. Reads stay open. Rotate the token with one update.

What can go wrong, and what catches it

RISK	PROB.	IMPACT	MITIGATION
Bedrock stays unauthorized for the account	medium	high	AWS Support case 178934068800881 is open. The graphs run unchanged on the Anthropic API with ANTHROPIC_API_KEY and GREN_BRIDGE=anthropic.
A feed changes its format or goes down	medium	medium	Each feed fails alone. The others continue. Recorded fixtures cover the tests. The first sweep uses a 400-day window.
A false negative in matching	low	high	Exact keys first, lexical with brand overlap second, then the model on the ambiguous pairs. The verifier sees the confirmed matches.
A false positive interrupts the household	medium	medium	The verifier can reject the plan. The budget limits interrupts. Snooze and decline are one tap.
A remedy sent twice	low	high	The side-effect node runs one time behind the gate. The frozen constraints forbid a retry without idempotency.
The household never answers	high	low	A decision expires after 72 hours. Nothing is sent without an approval. The digest keeps the record.
The demo token leaks	low	low	The token gates writes only. The daily cap bounds spend. A new token deploys in one update.

Why it is built this way

DECISION	RATIONALE
gren on Strands, not a bare agent loop	A YAML graph is inspectable, testable on a mock provider, and bounded. The engine, not the model, owns retries, budgets, and gates.
Code first, model second	Feed pagination, VIN decoding, UPC equality, and the sold window are code. Only the ambiguous pairs cost tokens. A quiet night is free.
An adversarial verifier with kill authority	A second prompt agrees too often. A verifier that can reject and repair the plan catches the plan that interrupts for nothing.
Gates as Strands interrupts with a checkpoint on disk	A household answers hours later. The process must be free to exit. Resume replays completed nodes with no new model calls.
A JSON store shaped like the DynamoDB design	One file per entity keeps the beta simple. The shape matches the plan, so the move to DynamoDB is a driver change.
EC2 and Caddy today, AgentCore next	The account quotas for AgentCore are 0 and only AWS Support can raise them. One instance gives a public HTTPS URL now. The AgentCore entrypoint is built.
Token gate and daily cap on the public demo	Reads are open for judges. Writes need the token. The cap protects the account whatever happens.
The Agent flow tab as a full trace workbench	An agent that acts at night must show its work in the morning. The workbench follows the execution UI of Kestra: overview, timeline, logs, replay.

What a new machine needs

ACCOUNT AND ACCESS

- An AWS account with a payment method. Region us-east-1.
- `aws login --profile guardian` (browser credentials, 12 h, renewable 90 days). No access keys in the repo.
- Bedrock authorization for the account. A new account answers Operation not allowed until AWS Support grants it.
- AgentCore quotas above 0 for the AgentCore path (agents, endpoints, versions). Support case only.

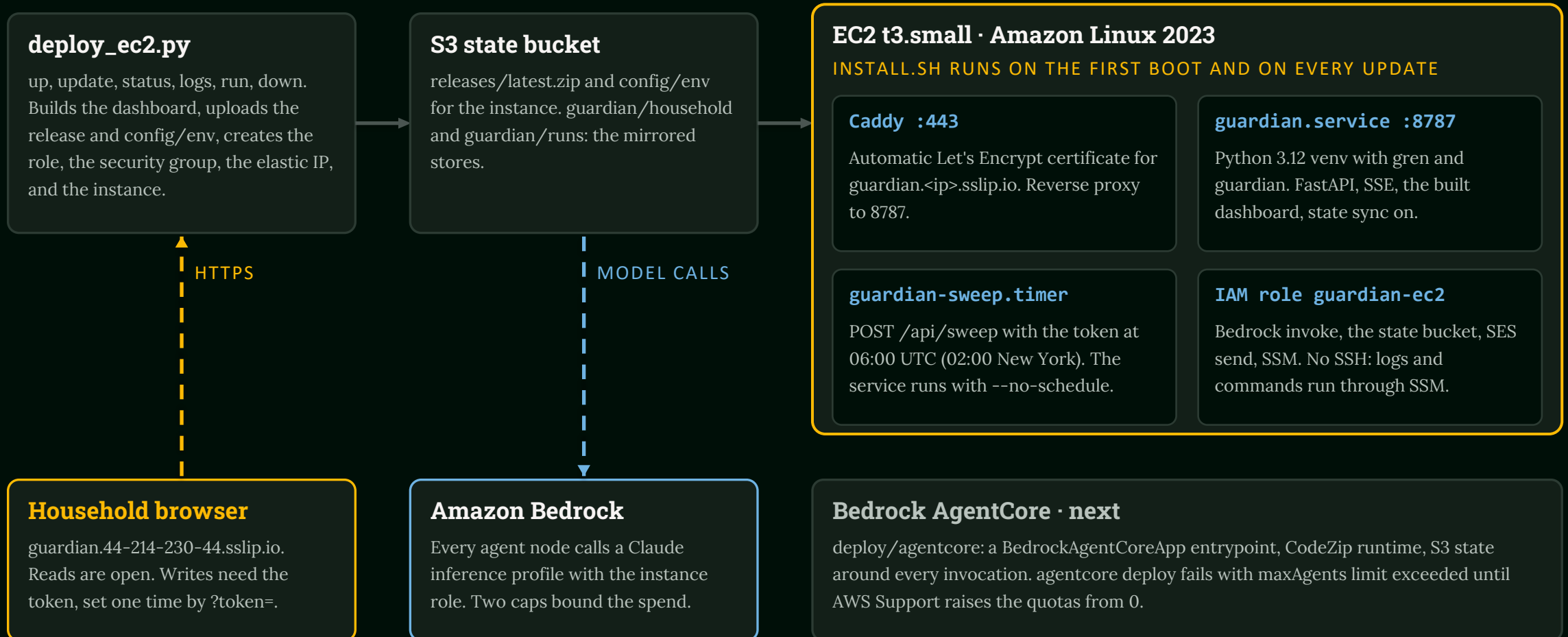
MODEL ACCESS

- Haiku 4.5: `global.anthropic.claude-haiku-4-5-20251001-v1:0`
- Sonnet 5: `global.anthropic.claude-sonnet-5`
- Opus 5: `global.anthropic.claude-opus-5`
- `GREN_BRIDGE` selects the provider: `bedrock`, `anthropic`, `claude-code`, or `mock`. `ANTHROPIC_API_KEY` is the fallback.

LOCAL TOOLING AND ENVIRONMENT

- Python 3.11 or later, Node 22, Chrome for the screenshots.
- `pip install -e ./gren[dev] -e .[dev]`, then `cd web && npm ci`.
- The `.env` file:
`GUARDIAN_DAILY_BUDGET_USD`,
`GUARDIAN_S3_BUCKET`,
`GUARDIAN_API_TOKEN`,
`GUARDIAN_SES_FROM`, `GUARDIAN_FEEDS`.
See `.env.example`.
- Tests: `GREN_ALLOW MOCK=1 python -m pytest -q` (48 tests, about 30 s).

One instance today, AgentCore next



From a clean account to a live URL

PREREQUISITES

- 1. Run `aws login --profile guardian`.
- 2. Create the private state bucket: `aws s3 mb s3://guardian-<account>-state`.
- 3. Fill `.env`: `AWS_PROFILE`, `AWS_REGION`, `GUARDIAN_S3_BUCKET`, `GUARDIAN_DAILY_BUDGET_USD`.
- 4. Run `python scripts/aws_check.py`. It makes one tiny Bedrock call.

DEPLOY

- 5. Run `python scripts/deploy_ec2.py up`. It builds the dashboard, uploads the release, and creates the role, the security group, the elastic IP, and the instance.
- 6. Wait for the health check. The first boot takes about 90 seconds.
- 7. Open the URL once with `?token=<GUARDIAN_API_TOKEN>`.
- 8. Push new code with `python scripts/deploy_ec2.py update`.

VERIFY AND OPERATE

- 9. `python scripts/deploy_ec2.py status`: the instance, the URL, and the provider.
- 10. `python scripts/deploy_ec2.py logs`: the boot log and the service journal over SSM.
- 11. Start a sweep from the dashboard. Watch it on the Agent flow tab.
- 12. `python scripts/deploy_ec2.py down --yes` removes everything but the bucket.

Done, blocked, next

DONE

- The nightly sweep and the intake graph run end to end on live feeds.
- The dashboard with the six pages and the trace workbench.
- The live deployment on EC2 with automatic HTTPS, S3 state sync, and the nightly timer.
- 48 tests on the mock provider, including the gate, snoozes, late approvals, and a verifier repair.

BLOCKED ON AWS

- Bedrock authorization for the account. Every model call answers Operation not allowed until AWS Support grants it.
- AgentCore Runtime. The account quotas are 0. The entrypoint is built and tested locally.
- SES delivery to a real sender. The sandbox needs a verified identity.

NEXT

- AgentCore Runtime, Memory, and Gateway.
- SMS by SNS with an origination number.
- Photo intake of receipts (the mailroom), grocery advisories, and class-action settlements: the Expanded tier of the plan.
- Cognito sign-in for more than one household.



The best output of the agent is silence. The primary metric of the product is how rarely it must speak to you.

`github.com/KrishGup/Recall-and-Warranty-Guardian`

`live: guardian.44-214-230-44.sslip.io`

APACHE-2.0 · STRANDS AGENTS SDK · EVERYDAY AGENTS TRACK